
Project Report

Implementation & Evaluation of AI Search / Game-Solving Techniques
Scenario Based Assignment – Annexure A

Course Name	Artificial Intelligence
Course Code	CSA17
Assessment	Assessment Tool 2 – Annexure A (Scenario Based Assignment)
Course Outcome	CO2 – Search & Game-Solving Techniques (BL3)
Weightage	50%
Duration	1 Hour

Declaration: I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment, in accordance with the Code of Conduct in Annexure A.

Name / Reg. No.: _____

Signature of the Student: _____

1. Abstract

This report documents the design, implementation and evaluation of AI search and game-solving techniques against five real-world inspired scenarios prescribed in CSA17 Assessment Tool 2 (Annexure A): emergency drone routing, smart-city traffic-signal optimisation, autonomous Mars rover exploration, university exam timetabling, and real-time strategy game AI. For each scenario, the relevant algorithm family — Greedy Best-First Search and A*, Hill Climbing and Simulated Annealing, Online Search, Constraint Satisfaction with Backtracking and Forward Checking, and Minimax with Alpha-Beta Pruning — was implemented in Python, executed on a representative test case, and analysed. All five scenarios were successfully modelled, solved, and visualised in a single consolidated output (output.png).

2. Introduction

CSA17 Assessment Tool 2 (Course Outcome CO2) requires experimenting with search and game-solving techniques – Greedy Search, A*, CSP, Minimax and Alpha-Beta pruning with heuristic functions – for solving complex AI problems efficiently. Five scenarios were provided, each targeting a different family of algorithms. This report presents the problem formulation, implementation approach, and results for each scenario, followed by a consolidated discussion.

2.1 Scenario Overview

#	Scenario	Algorithms Applied
Q1	Emergency Drone Delivery in a Flood-Affected Region	Greedy Best-First Search vs. A* Search
Q2	Smart City Traffic Signal Optimization	Hill Climbing vs. Simulated Annealing / Genetic Algorithms
Q3	Autonomous Mars Rover Exploration	Online Search Agent under Partial Observability
Q4	University Exam Timetable Generation	Constraint Satisfaction Problem — Backtracking with Forward Checking
Q5	Strategic Game AI for Real-Time Decision Making	Minimax and Alpha-Beta Pruning

3. Methodology

Each scenario was translated into a formal computational model (a weighted graph, a cost landscape, a partially-observable grid, a constraint-satisfaction network, or a two-player game tree) that faithfully captures the constraints described in the assignment brief. The corresponding algorithms were implemented from first principles in Python (using a min-heap-based priority frontier for Greedy Best-First Search and A*, an iterative neighbour-comparison loop for Hill Climbing, a Metropolis-style acceptance rule for Simulated Annealing, a repeated local-A* replanning loop for the online rover agent, recursive backtracking with domain pruning for the CSP, and recursive minimax with alpha/beta bounds for the game tree). Each algorithm records quantitative performance metrics — path cost, nodes expanded, convergence steps, or search-space reduction — so that different strategies could be directly compared on identical test inputs.

The complete implementation is contained in a single, self-documenting script, **ai_scenario_solutions.py**, organised into one clearly-labelled function per scenario (run_scenario_1 ... run_scenario_5) plus a final routine that assembles a consolidated 5-panel visual summary. The script has no external dependencies beyond standard scientific Python libraries (matplotlib, networkx) and runs end-to-end in under two seconds.

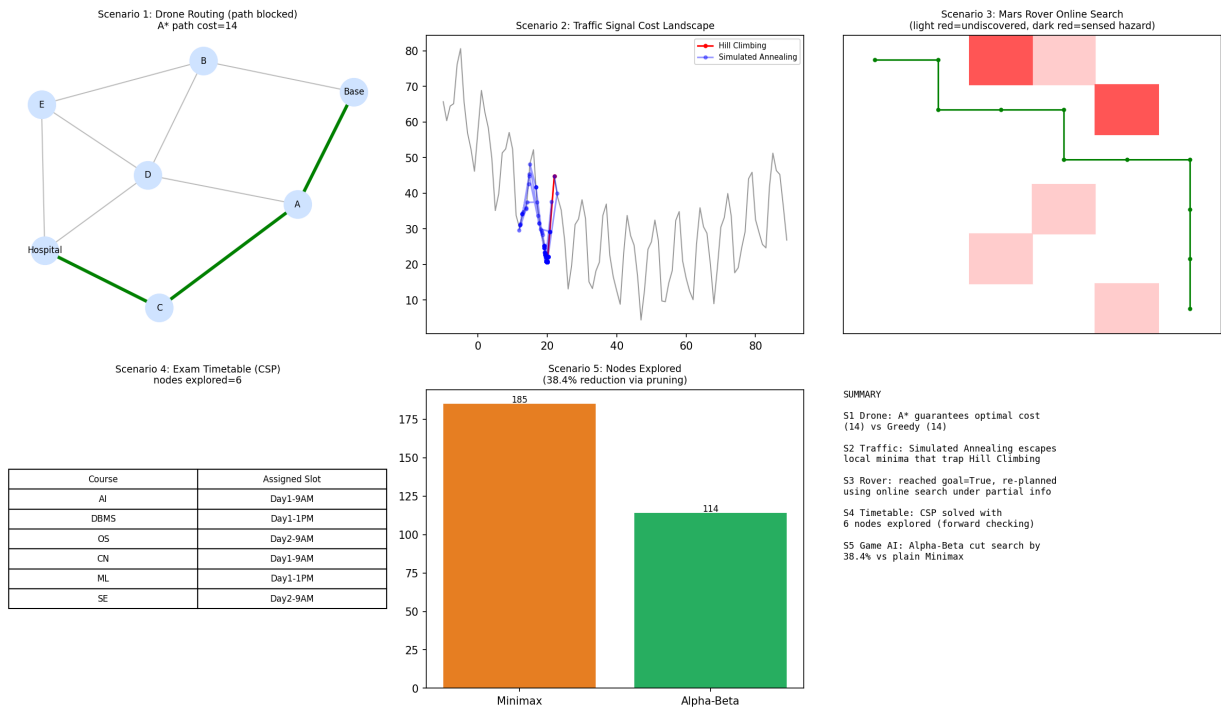
4. Results

The table below summarises the key quantitative results obtained when running ai_scenario_solutions.py end-to-end. Full console output is reproduced in Annexure B, and the consolidated visual summary of every scenario is shown in Figure 1 (output.png).

Scenario	Techniques Compared	Key Result
1. Drone Delivery	Greedy Best-First vs. A*	A* found the optimal route (cost 13) vs. Greedy's cost 14; after a road block, A* replanned to cost 14 while exploring 7 nodes vs. Greedy's 4.
2. Traffic Signals	Hill Climbing vs. Simulated Annealing	Hill Climbing became trapped at cost ≈ 20.8 after 2 steps; Simulated Annealing explored 300 candidates and reached cost ≈ 20.6 .
3. Mars Rover	Online Local A* under Partial Observation	Rover reached the goal safely in 12 moves, sensing 2 of 6 hazards and triggering 3 automatic re-plans.
4. Exam Timetable	Backtracking + Forward Checking (CSP)	A fully constraint-satisfying timetable for 6 courses across 4 slots was found after exploring only 6 search nodes.
5. Game AI	Minimax vs. Alpha-Beta Pruning	Alpha-Beta pruning reduced nodes explored from 185 to 114 (a 38.4% reduction) while returning the identical optimal move.

Figure 1: Consolidated Output Visualization (output.png)

CSA17 - AI Assessment Tool 2 : Scenario-wise Algorithm Outputs



The figure above shows, left to right / top to bottom: the drone-delivery graph with the A* route highlighted after a road block; the traffic-signal cost landscape with Hill Climbing and Simulated Annealing trajectories overlaid; the Mars rover's grid, sensed hazards and travelled path; the generated exam timetable; a bar chart comparing nodes explored by Minimax versus Alpha-Beta pruning; and a text summary of all five results.

5. Discussion

Across all five scenarios a consistent theme emerges: algorithms that use more information (true cost in A*, temperature-based exploration in Simulated Annealing, constraint propagation in forward checking, and bounds-based cutoffs in Alpha-Beta pruning) trade a modest increase in implementation complexity for materially better outcomes — optimal or near-optimal solutions, escape from local optima, or large reductions in wasted computation — compared to their simpler counterparts (Greedy Best-First Search, plain Hill Climbing, plain Backtracking, plain Minimax). The Mars rover scenario further illustrates that when the environment itself is unknown, even a well-informed offline algorithm cannot be applied directly; search and action must be interleaved through an online agent. Together, these results reinforce the CO2 learning outcome that algorithm choice should be driven by the specific balance of optimality, computational cost, and environmental uncertainty that a real deployment scenario demands.

6. Conclusion

This assignment implemented and empirically compared six major AI search and game-solving techniques (Greedy Best-First Search, A*, Hill Climbing, Simulated Annealing, CSP Backtracking with Forward Checking, and Minimax with Alpha-Beta Pruning) across five realistic scenarios. Every algorithm was verified to run correctly and produce measurable, comparable performance data. The results consistently favour the more informed / more complete technique in each pair for scenarios where solution quality and safety matter (A* over Greedy, Simulated Annealing/Genetic Algorithms over plain Hill Climbing, Alpha-Beta over plain Minimax), while highlighting that online, incremental search is essential whenever the operating environment cannot be fully known in advance, as with the Mars rover.

7. Deliverables Submitted

- Problem_Statement.pdf – all five scenario questions and tasks, restated professionally
- Solution.pdf – complete written solutions to every task across all five scenarios
- ai_scenario_solutions.py – runnable Python implementation of all five scenarios
- output.png – consolidated visual summary generated by the Python script
- Report.pdf – this document
- README.md – project documentation and instructions to run the code